

JAVASCRIPT ULTIMATE MEGA MASTER LIST (COMPLETE A-Z EDITION)

દરેક હિડન કન્સેપ્ટ, એન્જિન આર્કિટેક્ચર અને એડવાન્સ ટર્સનું સરળ ગુજરાતીમાં સિંગલ માસ્ટર લિસ્ટ

આ ડોક્યુમેન્ટમાં ઓનલાઈન ટ્યુટોરીયલ્સ અને ગૂગલ પરથી સામાન્ય રીતે મિસિંગ (ગાયબ) થતા અથવા અધૂરા મળતા તમામે તમામ JavaScript કન્સેપ્ટ્સને એક જ જગ્યાએ ઊંડાણપૂર્વક અને સરળ ગુજરાતીમાં સ્પષ્ટ કરવામાં આવ્યા છે.

૧. એડવાન્સ મેટા-પ્રોગ્રામિંગ અને મિસિંગ સિન્ટેક્સ (Meta-Programming & Hidden Syntax)

- **Decorators (@ સિન્ટેક્સ):** ક્લાસ, મેથડ, પ્રોપર્ટી કે પેરામીટરના લોજિકને મોડિફાય કે એક્સટેન્ડ કરવા માટે વપરાતું અલ્ટ્રા-મોડર્ન ફીચર. આ એક એવું ફંક્શન છે જે બીજા કોડને લપેટવા (wrap) અથવા ડેકોરેટ કરવા માટે વપરાય છે.
- **Generator Functions (function*):** આ એવા ખાસ ફંક્શન્સ છે જેને તમે રન થતી વખતે વચ્ચેથી અટકાવી શકો છો ('yield' કીવર્ડ દ્વારા) અને ભવિષ્યમાં ગમે ત્યારે ત્યાંથી જ ફરીથી શરૂ કરી શકો છો. આ 'Iterator' પ્રોટોકોલ પર કામ કરે છે.
- **Iterators & Iterables:** Iterable એટલે એવી કોઈ પણ વસ્તુ (જેમ કે Array, String) જેના પર લૂપ ફેરવી શકાય. Iterator એટલે એક એવો ઓબ્જેક્ટ જેની પાસે '.next()' મેથડ હોય અને તે લિસ્ટની આગામી વેલ્યુ રિટર્ન કરે.
- **Asynchronous Iterators (for await...of):** જ્યારે ડેટા સ્ટ્રીમ કે કલેક્શનના દરેક એલિમેન્ટ અસિંક્રોનસ રીતે (પ્રોમિસ ફોર્મેટમાં) મળતા હોય, ત્યારે તેના પર લૂપ ચલાવવા માટે આ સ્પેશિયલ સિન્ટેક્સ વપરાય છે.
- **Proxy Objects:** આ એક એવો કસ્ટમ સિક્રેટ એજન્ટ ઓબ્જેક્ટ છે જે તમારા ઓરિજિનલ ઓબ્જેક્ટની આગળ ઊભો રહી જાય છે. તમે ઓરિજિનલ ઓબ્જેક્ટમાં કોઈ પ્રોપર્ટી રીડ, રાઈટ કે ડીલીટ કરો તો પ્રોક્સી તેને અધવચ્ચે પકડીને ચેક કરે છે.
- **Reflect API:** જાવાસ્ક્રિપ્ટનું ગ્લોબલ ઓબ્જેક્ટ જે ઓબ્જેક્ટ્સની આંતરિક મેથડ્સ (જેમ કે પ્રોપર્ટી ચેક કરવી, ડીલીટ કરવી) ને સીધી રીતે ઓપરેટ કરવા માટે ક્લીન અને નવો રસ્તો આપે છે.
- **Symbols (Unique Tokens):** એક અનોખો અને ક્યારેય પણ ડુપ્લિકેટ ન થઈ શકે તેવો પ્રિમિટિવ ડેટા ટાઈપ. તેનો ઉપયોગ ઓબ્જેક્ટની અંદર એવી છુપી કી (Hidden Property Keys) બનાવવા થાય છે જેને કોઈ સામાન્ય લૂપ પકડી ન શકે.
- **Well-Known Symbols (Symbol.iterator વગેરે):** જાવાસ્ક્રિપ્ટના એન્જિનની અંદર બિલ્ટ-ઇન રહેલા ગુપ્ત સિમ્બોલ્સ. આનો ઉપયોગ કરીને તમે ઓરિજિનલ ઓબ્જેક્ટની અંદર લૂપ ચલાવવી કે ટાઈપ બદલવો જેવી નેટિવ સિસ્ટમને ઓવરરાઈડ કરી શકો છો.

૨. બ્રાઉઝર એન્જિન, કમ્પાઈલર અને મેમરી આર્કિટેક્ચર (Engine, Compiler & Memory Internals)

- **JIT (Just-In-Time) Compilation:** બ્રાઉઝરનું એન્જિન (જેમ કે V8) જાવાસ્ક્રિપ્ટના ઇન્ટરપ્રેટર કોડને રન ટાઈમ પર જ સીધો મશીન કોડમાં કમ્પાઈલ કરે છે જેથી કોડ સુપર-ફાસ્ટ સ્પીડમાં રન થાય.
- **Abstract Syntax Tree (AST):** જ્યારે તમે કોડ લખો, ત્યારે બ્રાઉઝરનું કમ્પાઈલર સૌથી પહેલાં તે કોડના શબ્દો અને વ્યાકરણને સમજીને મેમરીમાં એક પિરામિડ જેવું ટ્રી-સ્ટ્રક્ચર બનાવે છે, જેને AST કહેવાય છે.
- **Inline Caching (IC):** જો કોઈ પ્રોગ્રામમાં કોઈ ફંક્શન વારંવાર એક જ પ્રકારના ઓબ્જેક્ટની પ્રોપર્ટી એક્સેસ કરતું હોય, તો બ્રાઉઝર મેમરી લુકઅપ બાયપાસ કરીને તે પ્રોપર્ટીનું લોકેશન સીધું કેશમાં સેવ કરી લે છે.
- **Hidden Classes (Shapes):** જાવાસ્ક્રિપ્ટમાં ઓબ્જેક્ટ્સ ડાયનેમિક હોવા છતાં, બેકગ્રાઉન્ડમાં V8 એન્જિન સમાન પ્રકારની પ્રોપર્ટી ધરાવતા ઓબ્જેક્ટ્સ માટે એક 'હિડન ક્લાસ' બનાવે છે જેથી મેમરીમાં ડેટા ફાસ્ટ એક્સેસ થઈ શકે.
- **Memory Heap vs Call Stack:** Call Stack માં લાઈવ કોડ અને પ્રિમિટિવ વેરીએબલ્સ સ્ટોર થાય છે (LIFO ઓર્ડરમાં), જ્યારે મેમરી હીપ (Memory Heap) એ એક મોટો ખુલ્લો એરિયા છે જ્યાં મોટા ઓબ્જેક્ટ્સ અને ફંક્શન્સ અન-સ્ટ્રક્ચર્ડ રીતે સ્ટોર થાય છે.
- **Garbage Collection (Mark-and-Sweep):** જ્યારે મેમરી હીપમાં રહેલા કોઈ ઓબ્જેક્ટનો રેફરન્સ આખા પ્રોગ્રામમાંથી ખતમ થઈ જાય, ત્યારે જાવાસ્ક્રિપ્ટનું ગાર્બેજ કલેક્ટર રૂટથી ચેક કરીને તે નકામા ઓબ્જેક્ટને મેમરીમાંથી ઓટોમેટિકલી ડીલીટ કરી નાખે છે.
- **Memory Leaks:** એવી ભૂલો જેમાં કોડિંગની ખામીના કારણે નકામા ઓબ્જેક્ટ્સ મેમરીમાં જમા રહે છે અને ગાર્બેજ કલેક્ટર તેને ડીલીટ નથી કરી શકતું (દા.ત. અજાણતા બનેલા ગ્લોબલ વેરીએબલ્સ કે અધૂરા મુકેલા ટાઈમર્સ).

૩. માઈક્રો-ઓપ્ટિમાઈઝેશન અને બંડલર ટેકનોલોજી (Micro-Optimizations & Bundling)

- **Tree Shaking:** પ્રોડક્શન બિલ્ડ (ફાઇનલ કોડ) બનતી વખતે, મોડર્ન બંડલર્સ (Webpack, Vite) આખા કોડમાંથી એવા ફંક્શન્સ કે મોડ્યુલ્સને આપમેળે શોધીને હટાવી દે છે જે પ્રોગ્રામમાં ક્યાંય વપરાયા જ ન હોય.
- **Dead Code Elimination:** કમ્પાઈલર લેવલ પર થતું ઓપ્ટિમાઈઝેશન, જે એવા કોડ બ્લોક્સને કાયમ માટે રીમૂવ કરી દે છે જે ક્યારેય એક્ઝિક્યુટ થવાના જ નથી (જેમ કે `if (false) { ... }`).
- **Polyfills:** જ્યારે બ્રાઉઝર જૂનું હોય અને તેમાં જાવાસ્ક્રિપ્ટનું નવું ફીચર (દા.ત. `Promise.allSettled`) ઉપલબ્ધ ન હોય, ત્યારે તે ફીચરને મેન્યુઅલી બેકગ્રાઉન્ડ લોજિકથી જૂની સિન્ટેક્સમાં લખીને સપોર્ટેડ બનાવતા કોડને પોલીફિલ કહેવાય.
- **Transpilers (Babel):** આ એક એવું ટૂલ છે જે મોડર્ન ES6+ અથવા લેટેસ્ટ જાવાસ્ક્રિપ્ટ કોડને કન્વર્ટ કરીને જૂની ES5 સિન્ટેક્સમાં ફેરવે છે જેથી ઇન્ટરનેટ એક્સપ્લોરર કે જૂના બ્રાઉઝર્સમાં કોડ કેસ ન થાય.

- **Minification & Obfuscation:** કોડની ફાઇલ સાઇઝ ઘટાડવા વ્હાઇટ સ્પેસ અને કમેન્ટ્સ હટાવવાની પ્રોસેસ (Minification) તથા કોડને સિક્યોર કરવા માટે વેરીએબલ્સના નામ અથવા અક્ષરોમાં ફેરવવાની રીત (Obfuscation).

૪. સિક્યોરિટી અને એડવાન્સ વેબ આર્કિટેક્ચર (Security & Web Architecture)

- **CORS (Cross-Origin Resource Sharing):** બ્રાઉઝરની એક મહત્વની સિક્યોરિટી પોલિસી. તે નક્કી કરે છે કે એક વેબસાઇટ (દા.ત. domain-a.com) બીજી વેબસાઇટના (domain-b.com) API કે સર્વર પરથી ડેટા સેફલી લાવી શકે કે નહીં.
- **XSS (Cross-Site Scripting):** એક ખતરનાક વેબ એટેક જ્યાં હેકર્સ વેબસાઇટના ઇનપુટ બોક્સ અથવા યુઝરએલ દ્વારા પોતાનો દૂષિત જાવાસ્ક્રિપ્ટ કોડ યુઝરના બ્રાઉઝરમાં રન કરાવી દે છે અને કૂકીઝ કે સેશન ડેટા ચોરી લે છે.
- **CSRF (Cross-Site Request Forgery):** એવો એટેક જેમાં હેકર્સ યુઝરની જાણ બહાર તેના લોગ-ઇન સેશનનો દુરુપયોગ કરીને બીજા કોઈ મેલિસિયસ પેજ પરથી ઓરિજિનલ સર્વર પર ઓટોમેટિકલી ફોડ રિક્વેસ્ટ સબમિટ કરાવી દે છે.
- **Web Workers:** જાવાસ્ક્રિપ્ટ સિંગલ-થ્રેડેડ હોવાથી ભારે કેલ્ક્યુલેશન વખતે સ્ક્રીન હેંગ થઈ જાય છે. વેબ વર્કર્સ તમને મેઈન યુઝર ઇન્ટરફેસ (UI) ને ડિસ્ટર્બ કર્યા વગર બેકગ્રાઉન્ડમાં અલગ થ્રેડમાં ભારે લોજિક રન કરવાની સુવિધા આપે છે.

૫. એડવાન્સ અને છુપી જાવાસ્ક્રિપ્ટ ટર્મ્સ (Hidden Quirks & New ESNext)

- **Short-Circuit Evaluation:** તાર્કિક ઓપરેટર્સ (&& અને ||) નો શોર્ટ-કટ ઉપયોગ. જો ડાબી બાજુની શરત પરથી જ આખો જવાબ નક્કી થઈ જતો હોય, તો જાવાસ્ક્રિપ્ટ જમણી બાજુનો કોડ ચેક પણ નથી કરતી.
- **Temporal API:** વર્ષો જૂના, બગસથી ભરેલા ને અથરા Date() ઓબ્જેક્ટને કાયમ માટે રીપ્લેસ કરતું નવું ઓફિશિયલ ટાઈમ API, જે ટાઈમ-ઝોન કેલ્ક્યુલેશન સયોટ બનાવે છે.
- **Explicit Resource Management (using કીવર્ડ):** જ્યારે કોઈ ઓબ્જેક્ટ, ડેટાબેઝ કનેક્શન કે ફાઇલ હેન્ડલર પોતાના બ્લોક સ્કોપની બહાર જાય, ત્યારે તેની મેમરી આપમેળે નેટિવ લેવલ પર કલીન થઈ જાય તે માટેનો વેટેસ્ટ કીવર્ડ.
- **Math.sumPrecise():** જાવાસ્ક્રિપ્ટમાં $0.1 + 0.2 !== 0.3$ વાળી પૂર્ણાંક મેમરીની બહુ જૂની ભૂલને કાયમી માટે ફિક્સ કરતી નવી ઓફિશિયલ ગણિત પદ્ધતિ.
- **Variable Shadowing:** જ્યારે કોઈ ઇનર સ્કોપ (જેમ કે ફંક્શનની અંદર) માં બહારના સ્કોપ જેવા જ સમાન નામનો વેરીએબલ ડિક્લેર કરવામાં આવે, ત્યારે તે અંદરનો વેરીએબલ બહારના વેરીએબલને છુપાવી દે છે.